

Loops

Mahir Labib Dihan
Lecturer, CSE, BUET



Table of Contents

- | | |
|---------------------------------|--------------------------------------|
| 1. The Philosophy of Repetition | 5. Nested Loops |
| 2. The while Loop | 6. Jump Statements: break & continue |
| 3. The for Loop | 7. Some Random Questions |
| 4. The do-while Loop | 8. Summary & Best Practices |

The Philosophy of Repetition



Motivation

Scenario: Print "The sky is the limit!"

```
int main() {  
    printf("The sky is the limit!");  
}
```

Motivation

Scenario: Print "The sky is the limit!" *60 times*.

```
int main() {  
    printf("The sky is the limit!");  
    printf("The sky is the limit!");  
    printf("The sky is the limit!");  
    ... (Write this 60 times?)  
    printf("The sky is the limit!");  
    printf("The sky is the limit!");  
}
```



Loops / Repetition Statements

Definition:

- *Repetition statements* allow us to execute a statement multiple times.
- Often they are referred to as *loops*.

Scenario: Print "The sky is the limit!" *60 times*.

Without Loops:

- Write the logic once.
- Copy-paste it 60 times.

With Loops:

- Write the logic once.
- Tell the computer to repeat it 60 times.

Rule of thumb: If you are copy-pasting code more than twice, you likely need a loop.

Loops / Repetition Statements

Repetition statements allow us to execute a statement multiple times. They are referred to as *loops*.

Scenario: Print "The sky is the limit!" 60 times

Without Loops:

- `printf("The sky is the limit!");`
- `printf("The sky is the limit!");`
- ... (Write this 60 times?)
- `printf("The sky is the limit!");`

With Loops:

- Write the logic once.
- Tell the computer to repeat it 60 times.

Rule of thumb: If you are copy-pasting code more than twice, you likely need a loop.

Types of Loops in C

C has three kinds of repetition statements:

- the *while* loop
- the *for* loop
- the *do-while* loop

Key Insight: All three loops are interchangeable in functionality, but choosing the right one improves code readability

Types of Loops in C

Loop	Control Type	Best Use Case
while	Entry-controlled	Unknown iterations
do-while	Exit-controlled	Run at least once
for	Entry-controlled	Known iterations

Key Insight: All three loops are interchangeable in functionality, but choosing the right one improves code readability

The Three Essential Components

Every loop requires three components to work correctly:

1. **Initialization:** Set the starting point (e.g., $i = 0$)
2. **Condition:** Define when to continue/stop (e.g., $i < 10$)
3. **Update:** Change the loop variable (e.g., $i++$)

Common Bug

Forgetting the **update step** leads to an **infinite loop**!

The while Loop



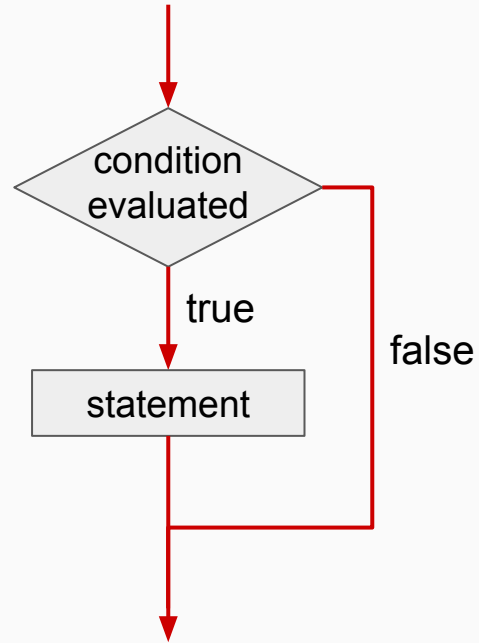
The while Loop

যতক্ষণ যদি

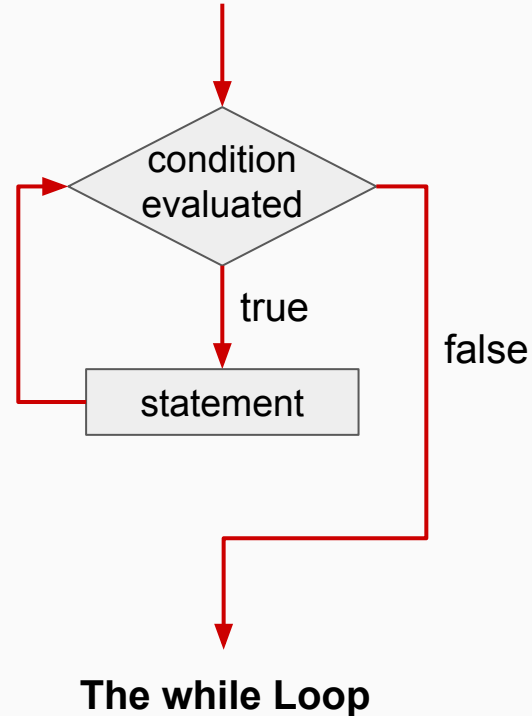
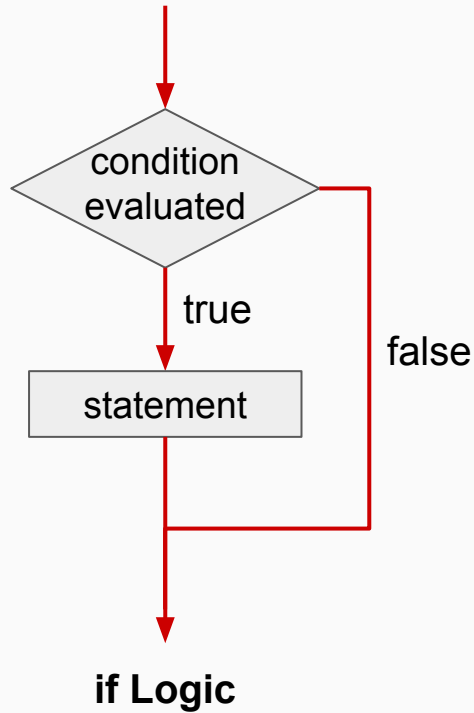
```
while if ( condition )  
    statement;
```

- **if** condition is satisfied execute the statement(s)
- **while** condition is satisfied execute the statement(s)

Flowchart: if Statement



Flowchart: while Loop



The `while` Loop: Syntax

- A while loop/statement has the following syntax:

```
while ( condition )  
    statement;
```

```
while ( condition ) {  
    statement1;  
    statement2;  
    ...  
}
```

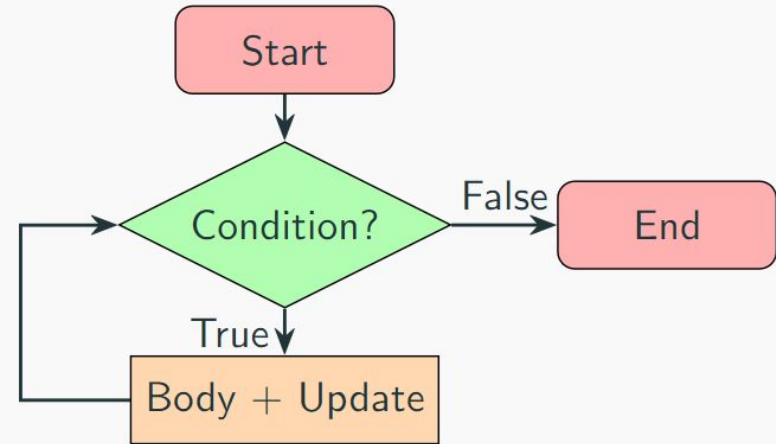
- If the *condition* is true, the *statement* or a *block of statements* is executed
- Then the condition is evaluated again, and if it is still true, the statement/block is executed again
- The statement/block is executed repeatedly until the condition becomes false

The while Loop: Syntax

The while loop checks the condition before executing the body.

Syntax

```
// Initialization BEFORE the loop  
int i = 0;  
  
while (condition) {  
    // Loop body  
    // Update step INSIDE  
    i++;  
}
```



Key Point: If condition is initially false, the loop body never runs.

Example: Positive Number

Task: Write a program that continuously asks for positive number as input.

```
int n;  
printf("Please enter a positive number: ");  
scanf("%d", &n);  
while (n < 0) {  
    printf ("Enter positive number, BE POSITIVE: ");  
    scanf("%d", &n);  
}
```

Output:

```
Please enter a positive number: -10  
Enter positive number, BE POSITIVE: -5  
Enter positive number, BE POSITIVE: -9  
Enter positive number, BE POSITIVE: 10
```

Example: Repetitive Print

Task: Print “The sky is the limit!” 60 times.

```
int n = 60;
int count = 0;
while (count < n)
{
    printf("The sky is the limit");
    count++;
}
```

Example: Repetitive Print

Task: Print “The sky is the limit!” n times. n will be user input.

```
int n;  
scanf("%d", &n);  
int count = 0;    // initialization  
while (count < n)  
{  
    printf("The sky is the limit");  
    count++;      // increment  
}
```

- If the condition of a `while` loop is false initially, the statement is never executed.
- Therefore, the body of a while loop will execute **zero or more** times.

The while Loop: Summary

Syntax

```
initialization;  
  
while (condition) {  
    statement;  
    increment;  
}
```

Example: The Countdown

```
#include<stdio.h>
int main() {
    int count = 5; // Initialization

    while (count > 0) { // Condition
        printf("%d", count);
        count--; // Update - CRITICAL !
    }
    printf("Blastoff!\n");
    return 0;
}
```

Output: 5 4 3 2 1 Blastoff!

Trace:

- count=5: Print 5, decrement to 4
- count=4: Print 4, decrement to 3
- ... continues until count=0, condition fails

Think First: What Happens Here?

Question: What is the output of this code?

```
int x = 10;
while(x > 0) {
    printf("%d ", x);
    x = x - 3;
}
printf("Done");
```

Answer: 10 7 4 1 Done

Trace:

- x=10: Print 10, x becomes 7
- x=7: Print 7, x becomes 4
- x=4: Print 4, x becomes 1
- x=1: Print 1, x becomes -2
- x=-2: Condition fails, exit loop

Common Mistake: The Semicolon Trap

Question: What is the output of this code?

```
int i = 0;
while (i < 5); // <-- Notice the semicolon!
{
    printf("%d ", i);
    i++;
}
```

Answer: Infinite Loop! (No output printed)

Trace:

- The semicolon after `while(i < 5)` acts as an empty body.
- The program gets stuck repeatedly checking `0 < 5` and doing nothing.
- The block `{ . . . }` below is treated as separate, unreachable code.

Example: Sum of Digits

Problem: Find the sum of digits of a number (e.g., $1234 \rightarrow 1+2+3+4 = 10$)

```
int num = 1234, sum = 0;

while (num > 0) {
    int digit = num % 10; // Extract last digit
    sum = sum + digit; // Add to sum
    num = num / 10; // Remove last digit
}

printf("Sum of digits: %d\n", sum);
```

Trace:

- num=1234: digit=4, sum=4, num=123
- num=123: digit=3, sum=7, num=12
- num=12: digit=2, sum=9, num=1
- num=1: digit=1, sum=10, num=0
- num=0: Condition fails, output 10

Example: Count Digits in a Number

Problem: How many digits are in a given number?

```
int num, count = 0;
printf("Enter a number: ");
scanf("%d", &num);

// Handle special case of 0
if (num == 0) {
    count = 1;
} else {
    int temp = num;
    if (temp < 0) temp = -temp ; // Handle negative

    while (temp > 0) {
        count++;
        temp = temp / 10;
    }
}
printf("Number of digits: %d\n", count);
```

Example: Reverse a Number

Problem: Reverse the digits (e.g., 1234 $\rightarrow$ 4321)

```
int num = 1234, reversed = 0;

while (num > 0) {
    int digit = num % 10;
    reversed = reversed * 10 + digit;
    num = num / 10;
}

printf("Reversed: %d\n", reversed);
```

Think: What happens with num = 1200?

Answer: Output is 21 (leading zeros are dropped in integers!)

Example: Finding GCD (Euclidean Algorithm)

Problem: Find GCD of two numbers using Euclid's method.

Algorithm: $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$ until $b = 0$

```
int a = 48, b = 18;

while (b != 0) {
    int temp = b;
    b = a % b;
    a = temp;
}

printf("GCD is %d\n", a);
```

Trace:

- $a=48, b=18$: $\text{temp}=18, b=48\%18=12, a=18$
- $a=18, b=12$: $\text{temp}=12, b=18\%12=6, a=12$
- $a=12, b=6$: $\text{temp}=6, b=12\%6=0, a=6$
- $b=0$: Exit. GCD = 6

The for Loop

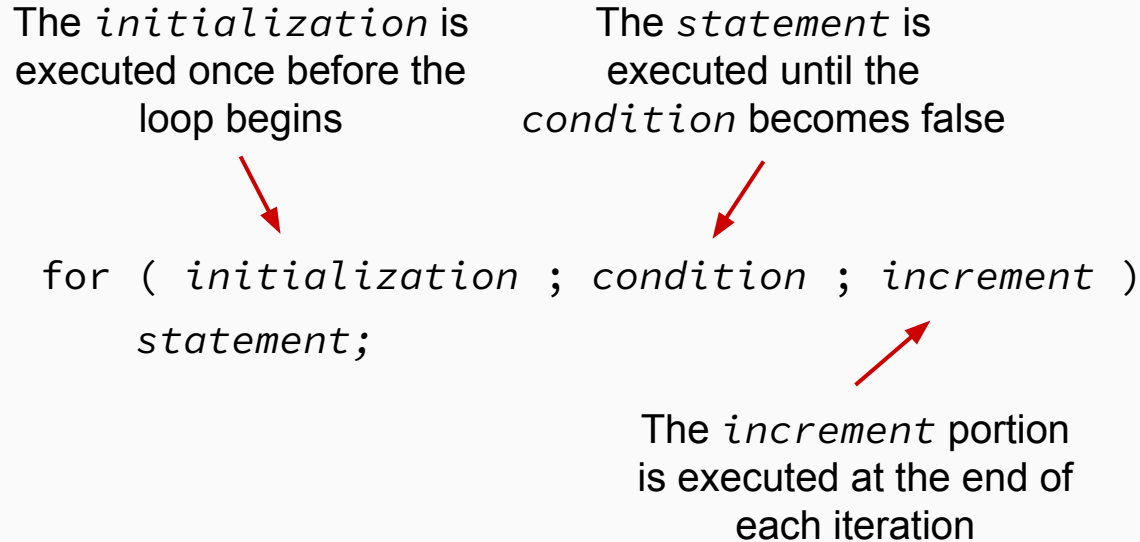


The for Loop: Syntax

- A for loop/statement has the following syntax:

The *initialization* is
executed once before the
loop begins

The *statement* is
executed until the
condition becomes false

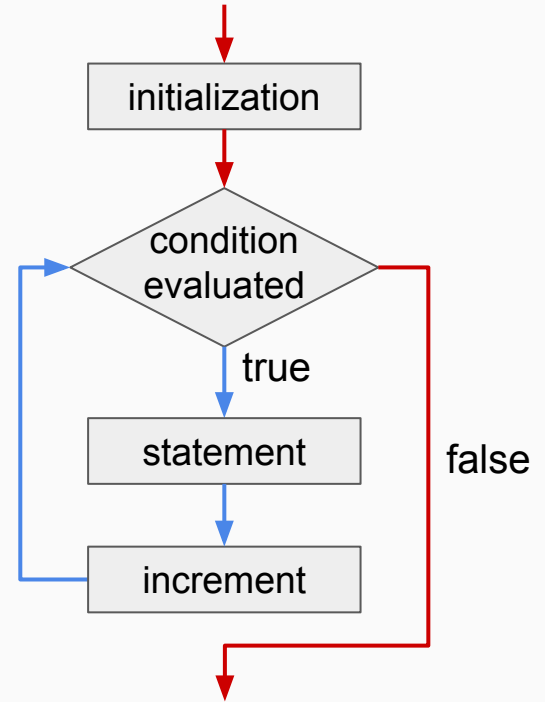
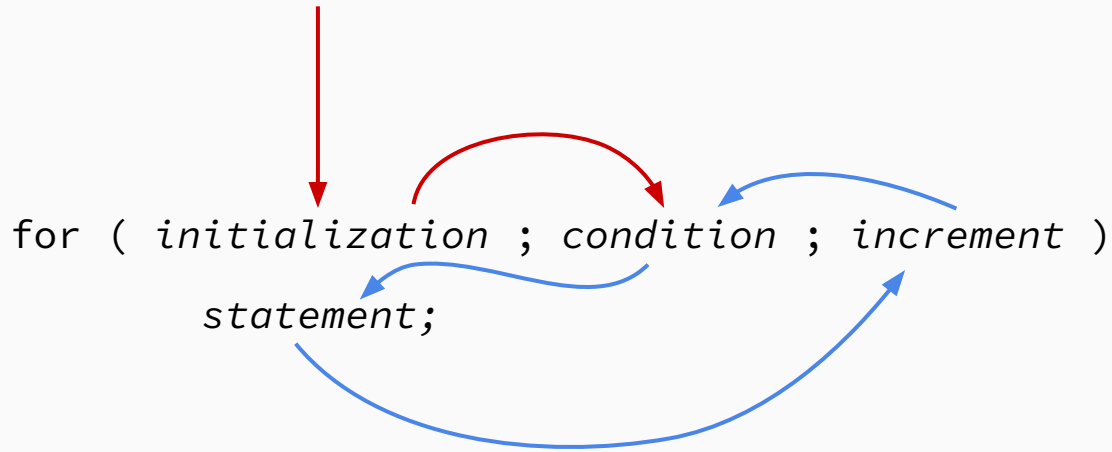


```
for ( initialization ; condition ; increment )  
    statement;
```

The diagram illustrates the syntax of a for loop. It shows the code `for ( initialization ; condition ; increment ) statement;`. Three red arrows point from explanatory text to the code: one from 'The initialization is executed once before the loop begins' to 'initialization', one from 'The statement is executed until the condition becomes false' to 'condition', and one from 'The increment portion is executed at the end of each iteration' to 'increment'.

The *increment* portion
is executed at the end of
each iteration

Flowchart: for Loop



The for Loop: Example

An example of a for loop:

```
for (count=1; count <= n; count++)  
    printf ("%d\n", count);
```

```
for (count=n; count >= 1; count--)  
    printf ("%d\n", count);
```

- The initialization section can be used to declare a variable
- Like a `while` loop, the condition of a `for` loop is tested prior to executing the loop body
- Therefore, the body of a `for` loop will execute *zero or more* times

The for Loop: Example

The increment section can perform any calculation

```
int num;  
for (num=100; num > 0; num -= 5)  
    printf ("%d\n", num);
```

- A for loop is well suited for executing statements a specific number of times that can be calculated or determined in advance

The for Loop: Anatomy

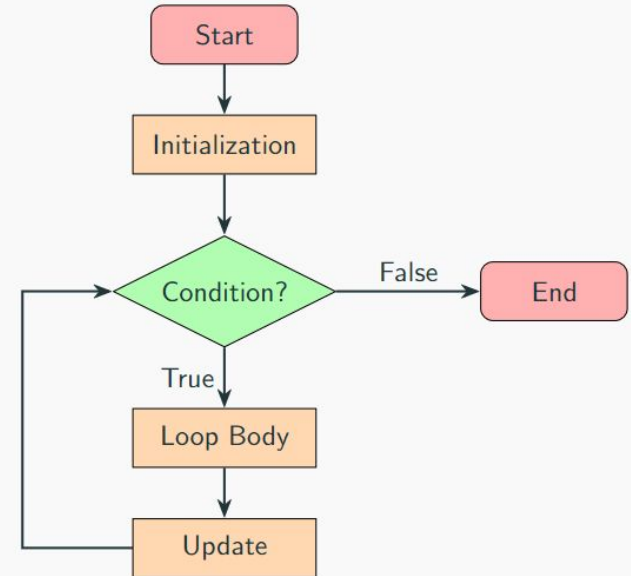
Best used when the **number of iterations is known** beforehand.

Syntax

```
for ( init ; condition ; update )  
{  
    // loop body  
}
```

Execution Order:

1. **Initialization:** Runs once at the start (e.g., $i = 0$)
2. **Condition:** Checked before each iteration (e.g., $i < 10$)
3. **Body:** Executes if condition is true
4. **Update:** Runs after the body (e.g., $i++$)
5. Go to step 2



Example: Print Numbers 1 to N

```
int n;  
printf("Enter N: ");  
scanf("%d", &n);  
  
for(int i = 1; i <= n; i++) {  
    printf("%d", i);  
}  
printf("\n");
```

For N=5: Output is 1 2 3 4 5

Trace:

- $i=1$: $1 \leq 5$? Yes. Print 1. i becomes 2.
- $i=2$: $2 \leq 5$? Yes. Print 2. i becomes 3.
- ... continues ...
- $i=6$: $6 \leq 5$? No. Exit loop.

Example: Sum of Series

Calculate $S = 1 + 2 + 3 + \dots + N$

```
int n, sum = 0;
printf("Enter N: ");
scanf("%d", &n);

for (int i = 1; i <= n ; i++) {
    sum = sum + i;
}

printf("Sum = %d\n", sum);
```

Example: Factorial Calculation

Calculate $N! = 1 \times 2 \times 3 \times \dots \times N$

```
int n ;  
long long factorial = 1; // Use long long for big numbers  
  
printf("Enter N: ");  
scanf("%d", &n);  
  
for (int i = 1; i <= n ; i++) {  
    factorial = factorial * i;  
}  
  
printf("%d! = %lld\n", n, factorial);
```

Note: Factorial grows very fast. 13! exceeds int range

Variations: Counting Down

```
// Print 10 to 1
for (int i = 10; i >= 1; i--) {
    printf("%d", i);
}
// Output : 10 9 8 7 6 5 4 3 2 1
```

```
// Print even numbers from 2 to 20
for (int i = 2; i <= 20; i += 2) {
    printf("%d", i);
}
// Output : 2 4 6 8 10 12 14 16 18 20
```

```
// Powers of 2: 1 , 2 , 4 , 8 , ...
for (int i = 1; i <= 1024; i *= 2) {
    printf("%d", i);
}
// Output : 1 2 4 8 16 32 64 128 256 512 1024
```

Advanced: Multiple Variables

You can have multiple variables in a for loop!

```
// Two variables moving towards each other  
for (int i = 0, j = 10; i < j; i++, j--) {  
    printf("i =%d, j =%d\n", i, j);  
}
```

Output:

i=0, j=10

i=1, j=9

i=2, j=8

i=3, j=7

i=4, j=6

Next iteration: i=5, j=5. Condition $i < j$ fails (5 < 5 is false)

Optional Parts: Infinite Loop

All three parts of the for loop header are optional!

```
// Infinite loop - all parts empty
for ( ; ; ) {
    printf("Forever...\n");
    // Use break to exit , or Ctrl + C
}
```

This is equivalent to `while(1) {...}`

```
// Initialization outside, update inside
int i = 0;
for ( ; i < 5 ; ) {
    printf ("%d", i);
    i++;
}
```

Think First

Question 1: What is the output?

```
for (int i = 0; i < 5; i++);  
    printf("%d", i);
```

Answer: Compilation ERROR! Variable `i` is not visible outside the loop.

Question 2: What about this?

```
int i;  
for(i = 0; i < 5; i++);  
    printf("%d", i);
```

Answer: Prints 5. The semicolon makes an empty loop body. Loop runs with `i=0, 1, 2, 3, 4`. When `i=5`, condition fails. Then we print `i`.

Critical: The Floating Point Trap

Never use floating point numbers as loop counters for exact equality!

```
for (float x = 0.0; x != 1.0; x += 0.1) {  
    printf("%f ", x);  
}
```

Why is this dangerous?

- Computers cannot represent 0.1 exactly in binary
- Adding 0.1 ten times might give 0.99999... or 1.00001
- The condition `x != 1.0` may remain true forever!

Safe Alternative:

```
for ( float x = 0.0; x < 1.0; x += 0.1) { ... }  
// OR use integer counter :  
for ( int i = 0; i < 10; i++) { float x = i * 0.1; ... }
```

The do-while Loop



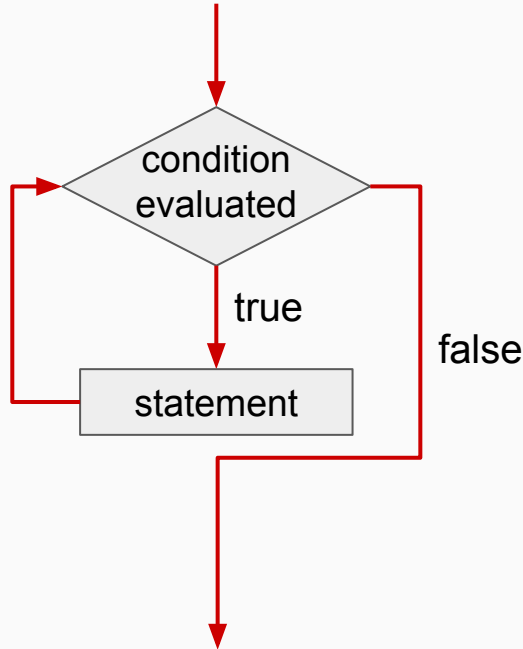
The do-while Loop: Syntax

- A do-while loop/statement has the following syntax:

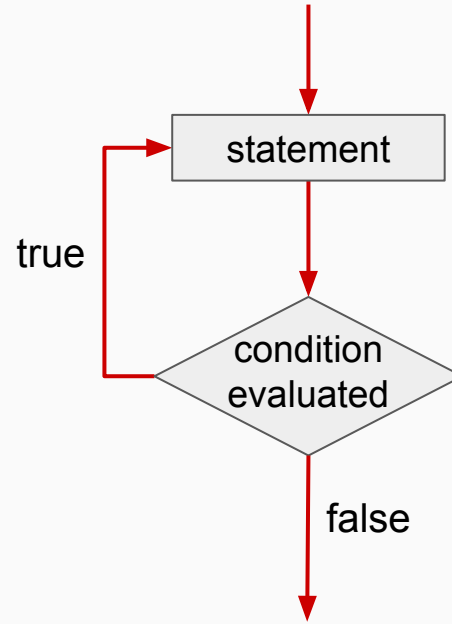
```
do {  
    statement;  
}  
while ( condition );
```

- The statement is executed once initially, and then the condition is evaluated
- The statement is executed repeatedly until the condition becomes false

Flowchart: do-while Loop



The while Loop



The do-while Loop

The do-while Loop: Example

An example of a do-while loop:

```
int count = 1;
do{
    printf("%d\n", count);
    count++;
} while (count <= 5);
```

- The body of a do-while loop is executed **at least once**

The do-while Loop: Example

while loop:

```
int n;
printf("Please enter a positive number: ");
scanf("%d", &n);
while (n < 0) {
    printf ("Please enter a positive number: ");
    scanf("%d", &n);
}
```

do-while loop:

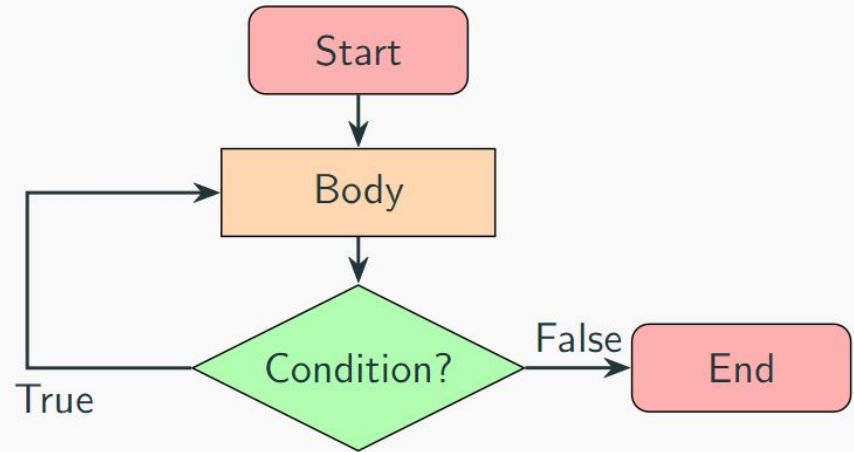
```
int n;
do {
    printf("Enter a positive number: ");
    scanf("%d",&n);
} while (n < 0);
```

The do-while Loop: Exit Controlled

The body is executed at least once, regardless of the condition.

Syntax

```
do {  
  // Loop body  
  // ( includes update )  
} while (condition);  
// -- Semicolon required !
```



Don't forget the semicolon after while(condition);

Use Case: Input Validation

do-while is perfect when you need to ask the user for input until they get it right

```
int number;

do {
    printf("Enter a positive number (1 -100): ");
    scanf("%d", &number);

    if (number <= 0 || number > 100) {
        printf("Invalid! Try again.\n");
    }
} while (number <= 0 || number > 100);

printf("You entered: %d\n", number);
```

Key Point: We MUST ask at least once before we can check!

Use Case: Menu-Driven Program

```
int choice;

do {
    printf("\n=== Calculator ===\n");
    printf("1. Add\n2. Subtract\n3. Exit\n");
    printf("Enter choice: ") ;
    scanf("%d", &choice);

    switch (choice) {
        case 1: printf("Addition selected\n"); break;
        case 2: printf("Subtraction selected\n"); break;
        case 3: printf("Goodbye!\n"); break;
        default: printf("Invalid choice!\n");
    }
} while (choice != 3);
```

Think First: while vs do-while

Question: What is the output of each?

Code A:

```
int x = 0;
while (x > 0) {
    printf("%d", x);
    x--;
}
printf("End");
```

Code B:

```
int x = 0;
do {
    printf ("%d", x);
    x--;
} while (x > 0);
printf("End");
```

Answer A: End (body never executes, condition false initially)

Answer B: 0 End (body executes once, then condition checked)

Comparison: while vs do-while

Situation	Recommended Loop
Entry Controlled	Exit Controlled
Checks condition first	Executes body first
May run 0 times	Runs at least 1 time
No semicolon at end	Ends with semicolon
Use when iteration might be 0	Use for menus, input validation

Mental Model:

- `while`: "Check, then do" (might not do anything)
- `do-while`: "Do, then check" (always does at least once)

Nested Loops



Nested Loops

- Similar to nested `if` statements, loops can be nested as well
- That is, the body of a loop can contain another loop
- For each iteration of the outer loop, the inner loop iterates completely

Nested Loops

What will be the output?

```
for(i=1; i <= 3; i++) { ← Outer Loop
    for(j=1; j <= 2; j++) { ← Inner Loop
        printf("Sky is the limit\n");
    }
    printf("The world is becoming smaller\n");
}
```

Nested Loops

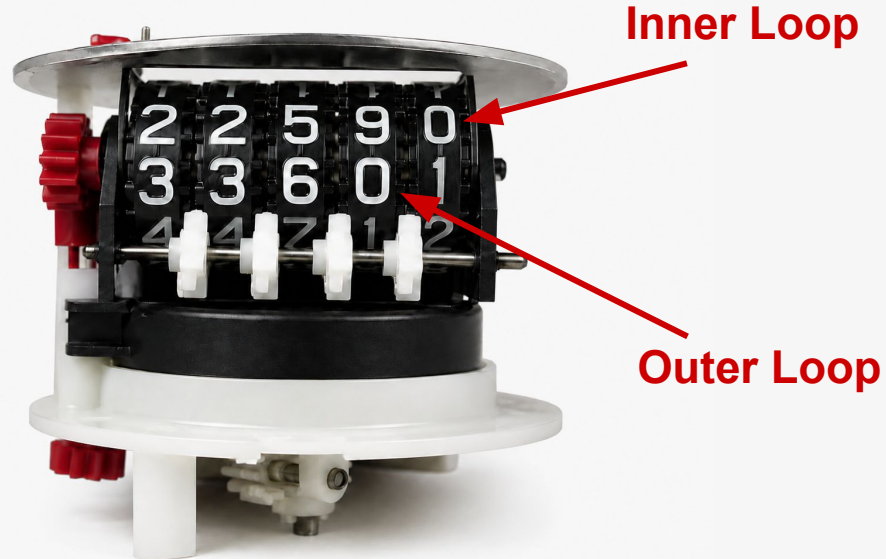
What will be the output?

```
for(i=1; i <= 3; i++) {  
    for(j=1; j <= 2; j++) {  
        printf("Sky is the limit\n");  
    }  
    printf("The world is becoming smaller\n");  
}
```

Output:

```
Sky is the limit  
Sky is the limit  
The world is becoming smaller  
Sky is the limit  
Sky is the limit  
The world is becoming smaller  
Sky is the limit  
Sky is the limit  
The world is becoming smaller
```

Analogy of Nested Loops



The Concept of Nesting

Analogy: A Clock

- The Second hand (Inner Loop) must make a full circle (60 ticks)...
- ..for the Minute hand (Outer Loop) to move just once.
- The Minute hand makes 60 ticks for the Hour hand to move once.

In Programming:

- The inner loop completes ALL its iterations for EACH iteration of the outer loop.
- Outer loop often controls Rows
- Inner loop often controls Columns

Total iterations: Outer count $\times$ Inner count

Example: Printing a Rectangle

```
for (int row = 1; row <= 3; row++) {  
    for (int col = 1; col <= 5; col++) {  
        printf ("* ");  
    }  
    printf("\n"); // Newline after each row  
}
```

Output:

* * * * *

* * * * *

* * * * *

Total stars: $3 \times 5 = 15$

Example: Multiplication Table

```
for (int i = 1; i <= 5; i++) {  
    for (int j = 1; j <= 5; j++) {  
        printf("%4d", i * j); // %4d for alignment  
    }  
    printf("\n");  
}
```

Output:

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20
5	10	15	20	25

Dependent Inner Loops: Right Triangle

What if the inner loop depends on the outer loop variable?

```
for (int i = 1; i <= 5; i++) {  
    for (int j = 1; j <= i; j++) { // j goes up to i!  
        printf("* ");  
    }  
    printf("\n");  
}
```

Output:

```
*  
* *  
* * *  
* * * *  
* * * * *
```

Pattern: Number Triangle

```
for (int i = 1; i <= 5; i++) {  
    for (int j = 1; j <= i ; j++) {  
        printf("%d", j);  
    }  
    printf("\n");  
}
```

Output:

```
1  
1 2  
1 2 3  
1 2 3 4  
1 2 3 4 5
```

Think First: Inverted Triangle

Question: What code produces this output?

```
* * * * *  
* * * *  
* * *  
* *  
*
```

Answer:

```
for (int i = 5; i >= 1; i--) {  
    for (int j = 1; j <= i ; j++) {  
        printf("* ");  
    }  
    printf("\n");  
}
```

Pyramid Pattern

Question: Print a centered pyramid

```
  *
 ***
*****
*****
*****
```

Analysis for row i (1 to n):

- Spaces before stars: $n - i$
- Number of stars: $2i - 1$

Pyramid Solution

```
int n = 5;
for (int i = 1; i <= n ; i++) {
    // Print spaces
    for (int s = 1; s <= n - i; s++) {
        printf(" ");
    }
    // Print stars
    for (int j = 1; j <= 2* i - 1; j++) {
        printf("* ");
    }
    printf("\n");
}
```


Question	My Answer
CSE 273 1. Write a program to print the pattern. <pre>1 4 6 9 11 13 17 19 21 23</pre>	<pre>#include <stdio.h> int main() { printf("1\n"); printf("4 6\n"); printf("9 11 13\n"); printf("17 19 21 23"); return 0; }</pre> 

Moral: This is why we learn loops!
(Hardcoding works... until the pattern changes to 100 rows)

Jump Statements: break & continue



The break and continue Statement

- Sometimes we need:
 - to skip some statements inside the loop (**continue**)
 - or terminate the loop immediately without checking the test condition (**break**)
- In such cases, break and continue statements are used

Control Flow Modifiers

Sometimes we need to alter the normal flow of a loop.

break Terminates the loop **immediately**. Control jumps to the statement after the loop body.

continue Skips the **rest of the current iteration**. Control jumps to:

- `for`: the update expression
- `while/do-while`: the condition check

Important: Both only affect the **innermost** loop they're in!

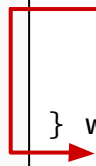
The break Statement

The break statement terminates the loop immediately when it is encountered

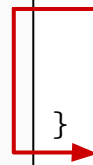
```
while (condition) {  
    // statements  
    if (condition to break) {  
        break;  
    }  
    // statements  
}
```



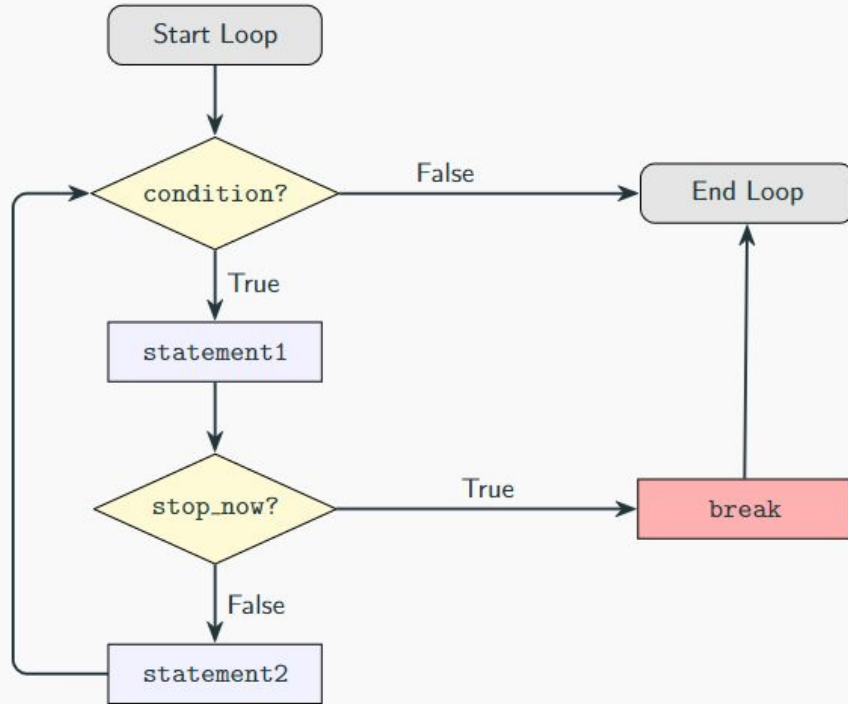
```
do {  
    // statements  
    if (condition to break) {  
        break;  
    }  
    // statements  
} while (condition);
```



```
for (init; condition; update) {  
    // statements  
    if (condition to break) {  
        break;  
    }  
    // statements  
}
```



Flowchart: The break Statement



Behavior:

- Terminates the loop immediately.
- Moves control to the statement just after the loop.

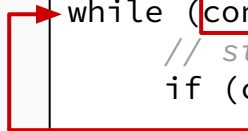
Example Logic:

```
while (condition) {  
    statement1;  
    if (stop_now) {  
        break; // Exit loop  
    }  
    statement2;  
}  
// Resumes here
```

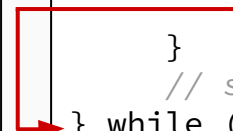
The continue Statement

The break statement terminates the loop immediately when it is encountered

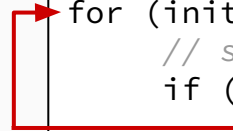
```
while (condition) {  
    // statements  
    if (condition) {  
        continue;  
    }  
    // statements  
}
```



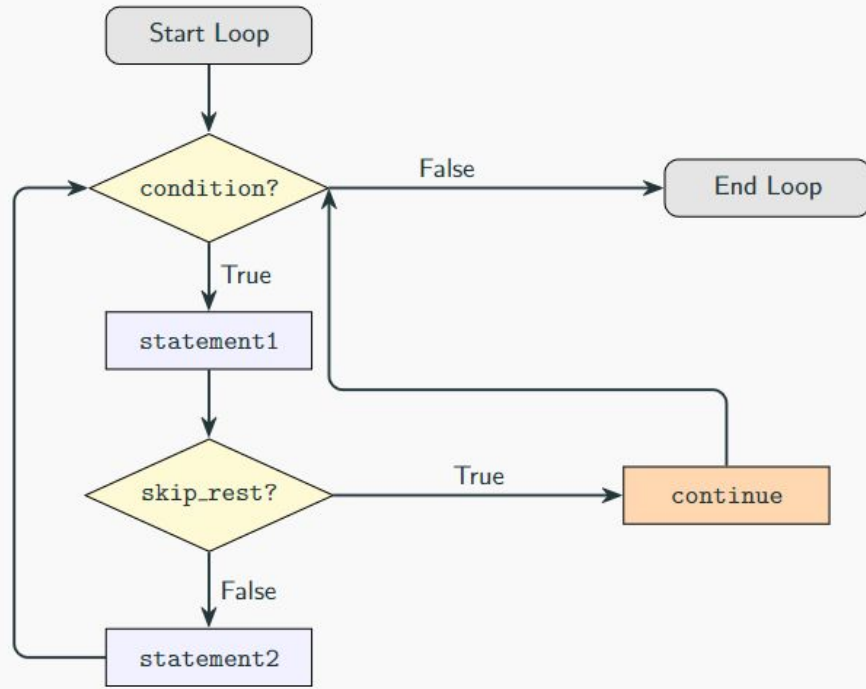
```
do {  
    // statements  
    if (condition to break) {  
        continue;  
    }  
    // statements  
} while (condition);
```



```
for (init; condition; update) {  
    // statements  
    if (condition to break) {  
        continue;  
    }  
    // statements  
}
```



Flowchart: The continue Statement



Behavior:

- Skips the rest of the current iteration (Part 2).
- Jumps back to evaluate the condition (or update).

Example Logic:

```
while (condition) {  
    statement1;  
    if (skip_rest) {  
        continue; // Next iter  
    }  
    statement2; // Skipped  
}
```


Example: break - Finding First Divisor

```
int n = 100;
printf("First divisor of %d (other than 1): ", n);

for (int i = 2; i <= n ; i++) {
    if (n % i == 0) {
        printf("%d\n", i);
        break; // Found it , stop searching!
    }
}
```

Output: First divisor of 100 (other than 1): 2

Without break: Would print ALL divisors (2, 4, 5, 10, 20, 25, 50, 100)

Example: break - Prime Checker

```
int n = 29, isPrime = 1;

for (int i = 2; i * i <= n ; i++) { // Only check up to sqrt(n)
    if (n % i == 0) {
        isPrime = 0;
        break; // Found a factor , no need to check more
    }
}

if (isPrime && n > 1)
    printf("%d is Prime\n", n);
else
    printf("%d is not Prime\n", n);
```

Efficiency: For $n=1000000$, we only check up to 1000 instead of 999999!

Example: continue - Skip Specific Values

Print 1 to 10, but skip multiples of 3:

```
for (int i = 1; i <= 10; i++) {  
    if (i % 3 == 0) {  
        continue; // Skip rest of body, go to i++  
    }  
    printf("%d ", i);  
}
```

Output: 1 2 4 5 7 8 10

(Skipped: 3, 6, 9)

Example: Sum of Non-Negative Numbers

Sum numbers until user enters negative (sentinel value):

```
int num, sum = 0;

while (1) { // Infinite loop
    printf("Enter a number (negative to stop): ");
    scanf("%d", &num);

    if (num < 0) {
        break; // Exit the loop
    }
    sum += num;
}

printf("Sum = %d\n", sum);
```

Danger: continue in while Loop

Be careful with continue in while loops!

```
int i = 0;
while (i < 5) {
    if (i == 2) {
        continue; // BUG: i is never incremented!
    }
    printf("%d", i);
    i++;
}
```

Result: Infinite loop! When $i=2$, continue skips $i++$.

Fix:

```
int i = 0;
while (i < 5) {
    if (i == 2) { i++; continue; } // Increment first!
    printf("%d ", i);
    i++;
}
```

Nested Loops: break Only Exits Innermost

```
for (int i = 1; i <= 3; i++) {  
    printf("Outer %d: ", i);  
    for (int j = 1; j <= 5; j++) {  
        if (j == 3) break; // Only breaks inner loop  
        printf("%d ", j);  
    }  
    printf("\n");  
}
```

Output:

Outer 1: 1 2

Outer 2: 1 2

Outer 3: 1 2

The outer loop continues! Each inner loop stops at j=3.

Breaking Out of Multiple Loops

Problem: How to exit all loops at once?

Solution 1: Use a flag variable

```
int found = 0;
for (int i = 0; i < 10 && !found; i++) {
    for (int j = 0; j < 10 && !found; j++) {
        if (arr[i][j] == target) {
            printf ("Found at (%d, %d)\n", i, j);
            found = 1; // This will stop both loops
        }
    }
}
```

Solution 2: Use goto (not recommended in general)

Some Random Questions



The Unsigned Trap

What happens with this countdown?

```
unsigned int i;  
for (i = 10; i >= 0; i--) {  
    printf("%u ", i);  
}
```

Infinite Loop!

- unsigned int can NEVER be negative
- When i is 0 and we do i--, it wraps to UINT MAX (≈ 4 billion)
- Condition $i \geq 0$ is ALWAYS true for unsigned

Fix:

```
for (unsigned int i = 10; i > 0; i--) { printf ("%u ", i); }  
printf("0 "); // Print 0 separately  
// OR use signed int
```

Pre vs Post Increment

```
int i = 0;
while (i++ < 3) {
    printf("%d ", i);
}
printf("Final: %d", i);
```

Output: 1 2 3 Final: 4

Trace:

1. Check $(0 < 3) = \text{True}$, THEN i becomes 1. Print 1.
2. Check $(1 < 3) = \text{True}$, THEN i becomes 2. Print 2.
3. Check $(2 < 3) = \text{True}$, THEN i becomes 3. Print 3.
4. Check $(3 < 3) = \text{False}$, THEN i becomes 4. Exit.

Empty Loop Bodies

What does this compute?

```
int n = 12345, count = 0;  
while (n > 0) n /= 10, count++;  
printf("%d", count);
```

Answer: 5 (counts number of digits)

The comma combines both statements into the condition area!

Another one:

```
int s = 0, i = 1;  
while (i <= 100) s += i++;  
printf("%d", s);
```

Answer: 5050 (sum of 1 to 100)

Problem: Prime Numbers in Range

Print all prime numbers between 2 and 50:

```
for (int n = 2; n <= 50; n++) {  
    int isPrime = 1;  
  
    for (int i = 2; i * i <= n; i++) {  
        if (n % i == 0) {  
            isPrime = 0;  
            break;  
        }  
    }  
  
    if (isPrime) {  
        printf("%d ", n);  
    }  
}
```

Output: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 50

Problem: Fibonacci Series

Print first N Fibonacci numbers: 0, 1, 1, 2, 3, 5, 8, 13, ...

```
int n = 10;
int a = 0, b = 1;

printf ("%d %d ", a, b);

for (int i = 3; i <= n; i++) {
    int next = a + b;
    printf("%d ", next);
    a = b;
    b = next;
}
```

Output: 0 1 1 2 3 5 8 13 21 34

Problem: Diamond Pattern

Goal: Print a diamond with $n=5$

```
  *
 * * *
* * * * *
* * * * * *
* * * * * * *
* * * * * * *
* * * * *
 * * * *
  * * *
   *
```

Hint: Combine pyramid (top half) with inverted pyramid (bottom half)

Summary & Best Practices



Loop Selection Guide

Situation	Recommended Loop
Known number of iterations	for
Unknown iterations, might be 0	while
Must execute at least once	do-while
Processing arrays	for
Reading until sentinel value	while with break
Menu-driven programs	do-while
Searching/finding first match	for/while with break

Common Mistakes to Avoid

1. **Infinite loops:** Forgetting update step
2. **Off-by-one errors:** $<$ vs $<=$
3. **Semicolon after condition:** `while(x);` or `for(...);`
4. **Floating point counters:** Never use for exact comparison
5. **Unsigned underflow:** `unsigned i; i >= 0` always true
6. **continue in while:** May skip increment
7. **Breaking wrong loop:** `break` only exits innermost
8. **Uninitialized variables:** Always initialize loop counters

Best Practices

1. **Initialize all variables** before using them
2. **Use meaningful names:** `row`, `col` instead of `i`, `j` when appropriate
3. **Keep loop bodies simple:** Extract complex logic into functions later
4. **Use braces always**, even for single statements
5. **Limit nesting depth:** More than 3 levels is hard to read
6. **Comment complex loops:** Explain what pattern is being generated
7. **Test boundary conditions:** 0 iterations, 1 iteration, max value

List of all Keywords in C Language

Keywords in C Programming			
auto	break	case	char
const	continue	default	do
double	else	enum	extern
float	for	goto	if
int	long	register	return
short	signed	sizeof	static
struct	switch	typedef	union
unsigned	void	volatile	while

Summary

1. **while:** Entry-controlled. Use when iterations unknown.
2. **for:** Best for counting. All parts (init, cond, update) in header.
3. **do-while:** Exit-controlled. Guarantees at least one execution.
4. **Nested loops:** Inner completes fully for each outer iteration. Think rows/columns.
5. **break:** Exit loop immediately.
6. **continue:** Skip to next iteration.

Acknowledgement

- Dr. Ashikur Rahman, Professor, CSE, BUET
- Mohammad Sadat Hossain, Lecturer, CSE, BUET